Java

Task	Command	Notes
Version manager	`sdkman`	`sdk install java 21-tem`
New project	`mvn archetype:generate`	or `gradle init`
Add dependency	edit `pom.xml`	or `build.gradle.kts`
Install deps	`mvn dependency:resolve`	or `gradle dependencies`
Build	`mvn package`	or `gradle build`
Run	`java -jar target/app.jar`	or `gradle run`
REPL	`jshell`	Java 9+
Test	`mvn test`	or `gradle test`; JUnit 5
Format	`google-java-format`	or Spotless plugin
Lint	—	IDE / SpotBugs / ErrorProne
LSP	`jdtls`	started by nvim and Emacs eglot

Basic Syntax

`var x = expr`	Local type inference
`final var x = expr`	Immutable local
`if / else if / else`	Conditional
`switch (x) { case ... }`	Switch statement
`case P -> expr`	Switch expression (14+)
`for (var x : xs)`	Enhanced for
`while / do-while`	Loops
`try / catch / finally`	Error handling
`throw new E("msg")`	Throw exception
`import static`	Static import

Types & Data

`record R(int x, String y)`	Record (16+)
`sealed interface I`	Sealed type (17+)
`permits A, B`	Permitted subtypes
`enum E { A, B, C }`	Enum
`interface I`	Interface
`class C extends B`	Inheritance
`implements I`	Implement interface
`Optional<T>`	Nullable wrapper
`@Override`	Override annotation

Modern Java (17+)

`case P when g ->`	Guarded pattern (21+)
`instanceof P p`	Pattern match (16+)
`""" text block """`	Multiline string (15+)
`yield expr`	Switch yield
`Stream.toList()`	Unmodifiable list (16+)
`SequencedCollection`	Ordered access (21+)
`.getFirst() / .getLast()`	First/last element (21+)

Generics

`class C<T>`	Generic class
`<T> T f(T x)`	Generic method
`<T extends Comparable>`	Upper bound
`List<? extends T>`	Covariant
`List<? super T>`	Contravariant
`List<?>`	Wildcard

Collections

`List.of(a, b, c)`	Immutable list
`Set.of(a, b)`	Immutable set
`Map.of(k1,v1, k2,v2)`	Immutable map
`new ArrayList<>()`	Mutable list
`new HashMap<>()`	Mutable map
`new HashSet<>()`	Mutable set
`Collections.unmodifiable*`	Unmodifiable view
`Deque` / `ArrayDeque`	Stack / queue

Streams

`xs.stream()`	Create stream
`.map(f)`	Transform
`.filter(p)`	Filter
`.flatMap(f)`	Flatten
`.reduce(id, f)`	Fold / reduce
`.collect(toList())`	Collect to list
`.collect(groupingBy(f))`	Group by key
`.collect(joining(","))`	Join strings
`.sorted(comparator)`	Sort
`.distinct()`	Unique
`.limit(n)` / `.skip(n)`	Take / drop
`.anyMatch(p)`	Existential
`.allMatch(p)`	Universal
`.findFirst()`	First as Optional
`Stream.of(a, b)`	From values
`IntStream.range(0, n)`	Int range

Functional Interfaces

`Function<T,R>`	T → R
`Predicate<T>`	T → boolean
`Consumer<T>`	T → void
`Supplier<T>`	() → T
`BiFunction<T,U,R>`	(T,U) → R
`Comparator<T>`	Ordering
`x -> expr`	Lambda
`ClassName::method`	Method reference

Testing (JUnit 5)

`@Test`	Test method
`assertEquals(e, a)`	Equality
`assertTrue(p)`	Boolean assertion
`assertThrows(E.class, ())`	Expect exception
`@BeforeEach` / `@AfterEach`	Setup / teardown
`@ParameterizedTest`	Parametric test
`@DisplayName("...")`	Test name

Error Handling

`Exception`	Checked exception
`RuntimeException`	Unchecked exception
`try-with-resources`	Auto-closeable
`throws E`	Declare checked
`e.getMessage()`	Error message
`e.getCause()`	Chained cause

Java — Concurrency & Standard Library

Concurrency — Basics

`Thread.start()`	Start thread
`Runnable`	Task (no return)
`Callable<T>`	Task (returns T)
`synchronized`	Mutual exclusion
`volatile`	Visibility guarantee
`Thread.sleep(ms)`	Sleep
`thread.join()`	Wait for completion

Concurrency — Executors

`Executors.newFixedThreadPool`
Fixed pool

`Executors.newVirtualThread*`
Virtual threads (21+)

`executor.submit(task)`	Submit task
`Future<T>`	Async result
`future.get()`	Block for result

CompletableFuture

`supplyAsync(() -> v)`	Async supplier
`.thenApply(f)`	Map result
`.thenCompose(f)`	FlatMap
`.thenAccept(c)`	Consume result
`.exceptionally(f)`	Handle error

`.thenCombine(other, f)`
Combine two

`allOf(a, b, c)`	Wait for all
`anyOf(a, b, c)`	First completed
`.join()`	Block (unchecked)

Virtual Threads (21+)

`Thread.ofVirtual().start(r)`
Start virtual thread

`Thread.startVirtualThread(r)`
Shorthand

`newVirtualThreadPerTask*()`
Executor factory

Concurrent Collections

`ConcurrentHashMap`	Thread-safe map
`CopyOnWriteArrayList`	Thread-safe list
`BlockingQueue`	Producer, consumer
`ConcurrentLinkedQueue`	Lock-free queue
`AtomicInteger / AtomicRef`	Lock-free vars

Optional

`Optional.of(v)`	Wrap non-null

`Optional.ofNullable(v)`
Wrap nullable

`Optional.empty()`	Empty optional
`.map(f)`	Transform if present
`.flatMap(f)`	Chain optionals
`.filter(p)`	Conditional keep
`.orElse(default)`	Default value
`.orElseThrow()`	Throw if empty
`.ifPresent(c)`	Consume if present
`.or(() -> other)`	Alternative Optional

I/O & Files

`Files.readString(path)`
Read file

`Files.writeString(path, s)`
Write file

`Files.readAllLines(path)`
Lines as list

`Files.list(dir)`	List directory
`Files.walk(dir)`	Recursive walk
`Files.exists(path)`	Check exists
`Files.copy(src, dst)`	Copy file
`Path.of("a", "b")`	Build path

Strings

`s.formatted(args)`	Format string (15+)
`"%s is %d".formatted(a,b)`	Printf-style
`String.join(",", xs)`	Join with separator
`s.split(",")`	Split to array
`s.strip()`	Trim whitespace
`s.contains(t)`	Substring test
`s.startsWith(t)`	Prefix test
`s.toCharArray()`	To char array
`s.chars()`	IntStream of chars

Common Annotations

`@Override`	Override check
`@Deprecated`	Mark deprecated
`@SuppressWarnings`	Suppress warning
`@FunctionalInterface`	Single abstract method
`@SafeVarargs`	Varargs safety

HTTP Client (11+)

```
HttpClient.newHttpClient()
    Create client
HttpRequest.newBuilder()
    Build request
.uri(URI.create(url))      Set URI
.GET() / .POST(body)       HTTP method
client.send(req,           Sync send
handler)
client.sendAsync(req,      Async send
h)
BodyHandlers.ofString()
    String response
```

Key Packages

`java.util`	Collections, Optional
`java.util.stream`	Stream API
`java.util.concurrent`	Concurrency
`java.nio.file`	File I/O
`java.net.http`	HTTP client
`java.time`	Date/time (8+)
`java.util.function`	Functional interfaces
`java.util.regex`	Regular expressions